Azure Machine Learning

TensorFlow, PyTorch

권기웅

1. TensorFlow

1.1 라이브러리 설치

1.1.1 TensorFlow

%pip install tensorflow

```
%pip install tensorflow

Collecting tensorflow
  Using cached tensorflow-2.18.0-cp310-cp310-manylinux_2_17_x86_64.manyl
Collecting absl-py>=1.0.0 (from tensorflow)
  Using cached absl_py-2.1.0-py3-none-any.whl.metadata (2.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Using cached flatbuffers-25.1.24-py2.py3-none-any.whl.metadata (875 by
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Using cached gast-0.6.0-py3-none-any.whl.metadata (1.3 kB)
Collecting google-pasta>=0.1.1 (from tensorflow)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Requirement already satisfied: libclang>=13.0.0 in /anaconda/envs/azurem
Collecting opt-einsum>=2.3.2 (from tensorflow)
  Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in /anaconda/envs/azureml_py310
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21
Requirement already satisfied: requests<3,>=2.21.0 in /anaconda/envs/azu
Requirement already satisfied: setuptools in /anaconda/envs/azureml_py31
Requirement already satisfied: six>=1.12.0 in /anaconda/envs/azureml_py3
Collecting termcolor>=1.1.0 (from tensorflow)
  Using cached termcolor-2.5.0-py3-none-any.whl.metadata (6.1 kB)
Requirement already satisfied: typing-extensions>=3.6.6 in /anaconda/env
Requirement already satisfied: wrapt>=1.11.0 in /anaconda/envs/azureml_p
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /anaconda/envs/azu
...
    Uninstalling numpy-1.23.5:
      Successfully uninstalled numpy-1.23.5
Successfully installed absl-py-2.1.0 astunparse-1.6.3 flatbuffers-25.1.2
Note: you may need to restart the kernel to use updated packages.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

1.1.2 NumPy

```
%pip install numpy==1.26.0

Collecting numpy==1.26.0
  Downloading numpy-1.26.0-cp310-cp310-manylinux_2_17_x86_64.many
Downloading numpy-1.26.0-cp310-cp310-manylinux_2_17_x86_64.manyli
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.2/18.2 MB 121.7 MB/s eta
Installing collected packages: numpy
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2
    Uninstalling numpy-2.0.2:
      Successfully uninstalled numpy-2.0.2
Successfully installed numpy-1.26.0
Note: you may need to restart the kernel to use updated packages.
```

1.2 재귀 제한 해제

```
import sys
sys.setrecursionlimit(10**7)
```

1.3 프로그램

```python
import numpy as np
import tensorflow as tf

xData = np.array([[80, 20], [95, 5], [10, 90], [90, 10], [5, 95]])
yData = np.array([[1, 0], [1, 0], [0, 1], [1, 0], [0, 1]])
label = ["액션", "조폭"]

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.Flatten(input_shape=(2, 1)),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dense(50, activation="relu"),
        tf.keras.layers.Dense(30, activation="relu"),
        tf.keras.layers.Dense(2, activation="softmax"),
    ]
)
o = tf.keras.optimizers.Adam(learning_rate=0.0001)
model.compile(optimizer=o, loss="categorical_crossentropy")

model.fit(xData, yData, epochs=100)

fight = int(input("싸움 : "))
yok = int(input("욕 : "))
predictData = np.array([[fight, yok]])
result = model.predict(predictData)
print(result)
print(label[result[0].argmax()])
```

[1] ✓ 11.2s

```
2025-02-05 09:58:42.358845: E external/local_xla/xla/str
WARNING: All log messages before absl::InitializeLog() i
E0000 00:00:1738749522.380103   58409 cuda_dnn.cc:8310]
E0000 00:00:1738749522.388143   58409 cuda_blas.cc:1418]
2025-02-05 09:58:42.417206: I tensorflow/core/platform/c
To enable the following instructions: AVX2 AVX512F FMA,
/anaconda/envs/azureml_py310_sdkv2/lib/python3.10/site-p
  super().__init__(**kwargs)
2025-02-05 09:58:45.176492: E external/local_xla/xla/str
Epoch 1/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 977ms/step - loss: 3.3608
Epoch 2/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 3.2046
```

1.4 모델 만들기

```python
import numpy as np
import tensorflow as tf
import mlflow

mlflow.start_run()

xData = np.array([[80, 20], [95, 5], [10, 90], [90, 10], [5, 95]])
yData = np.array([[1, 0], [1, 0], [0, 1], [1, 0], [0, 1]])
label = ["액션", "조폭"]

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.Flatten(input_shape=(2, 1)),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dense(50, activation="relu"),
        tf.keras.layers.Dense(30, activation="relu"),
        tf.keras.layers.Dense(2, activation="softmax")
    ]
)
o = tf.keras.optimizers.Adam(learning_rate=0.0001)
model.compile(optimizer=o, loss="categorical_crossentropy")

model.fit(xData, yData, epochs=100)

fight = 30
yok = 20
predictData = np.array([[fight, yok]])
result = model.predict(predictData)
print(label[result[0].argmax()])

mlflow.tensorflow.save_model(
    model=model,
    path="kwon_tf_model",
)

mlflow.end_run()
```

[2]

```
/anaconda/envs/azureml_py310_sdkv2/lib/python3.10/
  from google.protobuf import service as _service
Epoch 1/100
/anaconda/envs/azureml_py310_sdkv2/lib/python3.10/
  super().__init__(**kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 821ms/step - loss: 5.1724
Epoch 2/100
```

2. PyTorch

2.1 라이브러리 설치

```
%pip install torch
```

```
%pip install torch
```

```
[2]
...
Collecting torch
  Downloading torch-2.6.0-cp310-cp310-manylinu
Requirement already satisfied: filelock in /an
Requirement already satisfied: typing-extensio
Requirement already satisfied: networkx in /an
Requirement already satisfied: jinja2 in /anac
Requirement already satisfied: fsspec in /anac
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (f
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-
Collecting nvidia-cuda-runtime-cu12==12.4.127
  Downloading nvidia_cuda_runtime_cu12-12.4.12
Collecting nvidia-cuda-cupti-cu12==12.4.127 (f
  Downloading nvidia_cuda_cupti_cu12-12.4.127-
Collecting nvidia-cudnn-cu12==9.1.0.70 (from t
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-n
Collecting nvidia-cublas-cu12==12.4.5.8 (from
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-
Collecting nvidia-cufft-cu12==11.2.1.3 (from t
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-n
```

2.2 프로그램

```python
import numpy as np
import torch
from torch import nn

xData = np.array([[80, 20], [95, 5], [10, 90], [90, 10], [5, 95]], dtype=np.float32)
yData = np.array([[1, 0], [1, 0], [0, 1], [1, 0], [0, 1]], dtype=np.float32)
label = ["액션", "조폭"]
xData = torch.from_numpy(xData)
yData = torch.from_numpy(yData)

device = (
    torch.accelerator.current_accelerator().type
    if torch.accelerator.is_available()
    else "cpu"
)

class KwonNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.nn = nn.Sequential(
            nn.Linear(2 * 1, 100),
            nn.ReLU(),
            nn.Linear(100, 50),
            nn.ReLU(),
            nn.Linear(50, 30),
            nn.ReLU(),
            nn.Linear(30, 2),
        )

    def forward(self, x):
        logits = self.nn(x)
        return logits


model = KwonNN().to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
```

```python
model.train()
for t in range(100):
    for i, x in enumerate(xData):
        y = yData[i]
        x = x.to(device)
        y = y.to(device)

        pred = model(x)
        loss = loss_fn(pred, y)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        print(loss.item())

model.eval()
fight = int(input("싸움 : "))
yok = int(input("욕 : "))
predictData = np.array([[fight, yok]], dtype=np.float32)
predictData = torch.from_numpy(predictData)
result = model(predictData)
result = nn.Softmax()(result)
result = result.argmax().item()
result = label[result]
print(result)
```

[4]

```
/anaconda/envs/azureml_py310_sdkv2/lib/python3.10/si
  return self._call_impl(*args, **kwargs)
0.3557509481906891
0.339968204498291
0.628268837928772
```

2.3 모델 만들기

```python
import torch
from torch import nn
import numpy as np
import mlflow

mlflow.start_run()

xData = np.array([[80, 20], [95, 5], [10, 90], [90, 10], [5, 95]], dtype=np.float32)
yData = np.array([[1, 0], [1, 0], [0, 1], [1, 0], [0, 1]], dtype=np.float32)
label = ["액션", "조폭"]
xData = torch.from_numpy(xData)
yData = torch.from_numpy(yData)

device = (
    torch.accelerator.current_accelerator().type
    if torch.accelerator.is_available()
    else "cpu"
)

class KwonNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.nn = nn.Sequential(
            nn.Linear(2 * 1, 100),
            nn.ReLU(),
            nn.Linear(100, 50),
            nn.ReLU(),
            nn.Linear(50, 30),
            nn.ReLU(),
            nn.Linear(30, 2),
        )

    def forward(self, x):
        logits = self.nn(x)
        return logits


model = KwonNN().to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
```

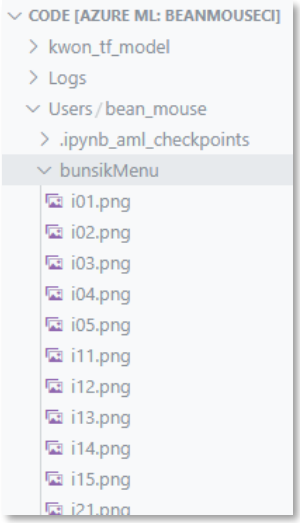
```python
model.train()
for t in range(10):
    for i, x in enumerate(xData):
        y = yData[i]
        x = x.to(device)
        y = y.to(device)

        pred = model(x)
        loss = loss_fn(pred, y)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

model.eval()
fight = 30
yok = 80
predictData = np.array([[fight, yok]], dtype=np.float32)
predictData = torch.from_numpy(predictData)
result = model(predictData)
result = nn.Softmax()(result)
result = result.argmax().item()
result = label[result]
print(result)

mlflow.pytorch.save_model(
    pytorch_model=model,
    path="kwon_pt_model",
)

mlflow.end_run()
```

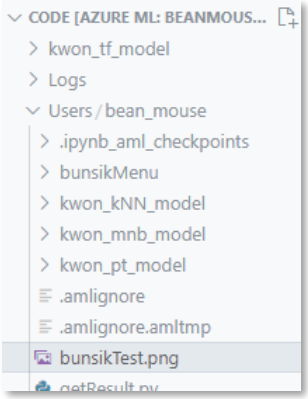
[26]

```
조폭
2025/02/05 09:40:17 INFO mlflow.tracking._tracking
2025/02/05 09:40:17 INFO mlflow.tracking._tracking
```

2.4 이미지 인식

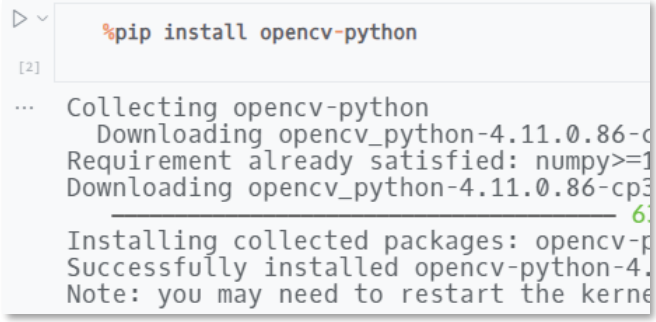
2.4.1 이미지 업로드 하고





2.4.2 라이브러리 설치

```
%pip install opencv-python
```



2.4.3 프로그램

```python
import torch
from torch import nn
import numpy as np
import cv2
import os

def getImg(folder, w, h):
    datas = []
    for f in os.listdir(folder):
        data = cv2.imread(folder + "/" + f, cv2.IMREAD_GRAYSCALE)
        data = cv2.resize(data, (w, h))
        datas.append(data)
    return np.array(datas, dtype=np.float32)


xData = getImg("bunsikMenu", 100, 50)
yData = np.array(
    [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4]
)
label = ["떡볶이", "오뎅", "김밥", "튀김", "순대"]
xData = torch.from_numpy(xData)
xData = nn.Flatten()(xData)
yData = torch.from_numpy(yData)
yData = torch.nn.functional.one_hot(yData, num_classes=5)
yData = yData.type(torch.float32)

device = (
    torch.accelerator.current_accelerator().type
    if torch.accelerator.is_available()
    else "cpu"
)

class KwonNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.nn = nn.Sequential(
            nn.Linear(50 * 100, 100),
            nn.ReLU(),
            nn.Linear(100, 50),
            nn.ReLU(),
            nn.Linear(50, 30),
            nn.ReLU(),
            nn.Linear(30, 5),
        )

    def forward(self, x):
        logits = self.nn(x)
        return logits


model = KwonNN().to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
```

```python
model.train()
for t in range(10):
    for i, x in enumerate(xData):
        y = yData[i]
        x = x.to(device)
        y = y.to(device)

        pred = model(x)
        loss = loss_fn(pred, y)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

model.eval()

f = cv2.imread("bunsikTest.png", cv2.IMREAD_GRAYSCALE)
f = cv2.resize(f, (100, 50))
f = np.array([f], dtype=np.float32)
predictData = torch.from_numpy(f)
predictData = nn.Flatten()(predictData)
result = model(predictData)
result = nn.Softmax()(result)
result = result.argmax().item()
result = label[result]
print(result)
```

[9] ✓ 0.8s

```
순대
```

2.5 CNN

2.5.1 준비

```python
from random import random
import torch
from torch.utils.data import Dataset, DataLoader
from torch import accelerator, nn, optim
import numpy as np
import cv2
import os

##############################
device = accelerator.current_accelerator().type if accelerator.is_available() else "cpu"
```

2.5.2 데이터셋

```python
##############################
def getImg(folder, w, h, yData):
    train_datas = []
    train_labels = []
    test_datas = []
    test_labels = []
    for i, f in enumerate(os.listdir(folder)):
        data = cv2.imread(folder + "/" + f, cv2.IMREAD_COLOR)
        data = cv2.resize(data, (w, h))
        if random() < 0.3:
            test_datas.append(np.array(data, dtype=np.float32))
            test_labels.append(yData[i])
        else:
            train_datas.append(np.array(data, dtype=np.float32))
            train_labels.append(yData[i])
    return train_datas, train_labels, test_datas, test_labels


class KwonDataSet(Dataset):
    def __init__(self, data, label, numClasses):
        super().__init__()
        self.datas = torch.from_numpy(np.array(data))
        self.datas = self.datas.permute(0, 3, 1, 2)
        self.labels = torch.from_numpy(np.array(label))
        self.labels = nn.functional.one_hot(self.labels, num_classes=numClasses)
        self.labels = self.labels.type(torch.float32)

    def __len__(self):
        return len(self.datas)

    def __getitem__(self, index):
        return self.datas[index], self.labels[index]


label = ["떡볶이", "오뎅", "김밥", "튀김", "순대"]
y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4]
train_datas, train_labels, test_datas, test_labels = getImg("bunsikMenu", 100, 50, y)

trainDataset = KwonDataSet(train_datas, train_labels, 5)
testDataset = KwonDataSet(test_datas, test_labels, 5)
trainDataloader = DataLoader(trainDataset, 5, shuffle=True)
testDataloader = DataLoader(testDataset, 5, shuffle=True)
```

2.5.3 CNN

```python
##############################
class KwonNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.nn = nn.Sequential(
            nn.Conv2d(3, 1000, 5),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(1000, 500, 5),
            nn.ReLU(),
            nn.Conv2d(500, 100, 5),
            nn.ReLU(),
            nn.Conv2d(100, 50, 5),
            nn.ReLU(),
        )
        self.f = nn.Flatten()
        self.nn2 = nn.Sequential(nn.Linear(19800, 10), nn.ReLU(), nn.Linear(10, 5))

    def forward(self, x):
        logits = self.nn(x)
        logits = self.f(logits)
        logits = self.nn2(logits)
        return logits


model = KwonNN().to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.000001)
```

2.5.4 훈련 & 테스트

```
##############################
model.train()
epoch = 10
for t in range(epoch):
    for batch, (X, y) in enumerate(trainDataloader):
        X = X.to(device)
        y = y.to(device)

        pred = model(X)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        print(loss.item())

##############################
model.eval()
size = len(testDataloader.dataset)
correct = 0
with torch.no_grad():
    for X, y in testDataloader:
        X = X.to(device)
        y = y.to(device)

        pred = model(X)
        print(pred.argmax(1))
        print(y.argmax(1))
        correct += (pred.argmax(1) == y.argmax(1)).type(torch.float32).sum().item()
    print(correct / size)
```

2.5.5 예측

```
##############################
f = cv2.imread("bunsikTest.png", cv2.IMREAD_COLOR)
f = cv2.resize(f, (100, 50))
f = np.array([f], dtype=np.float32)
pred_data = torch.from_numpy(f)
pred_data = pred_data.permute(0, 3, 1, 2)
result = model(pred_data)
result = nn.Softmax(dim=1)(result)
print(result)
result = result.argmax().item()
result = label[result]
print(result)
```

[5] ✓ 48.1s

```
2.0464344024658203
1.573398232460022
2.053532361984253
1.8629789352416992
1.3963615894317627
1.5688632726669312
```